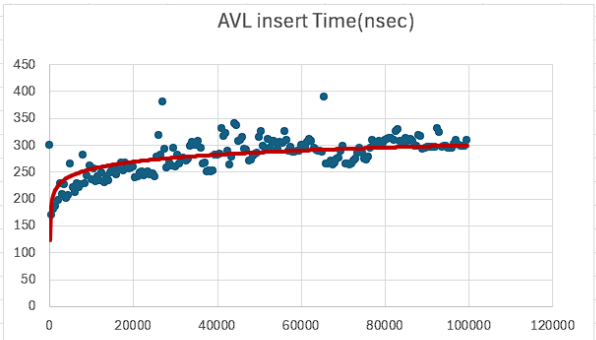
Avramenko Maksym

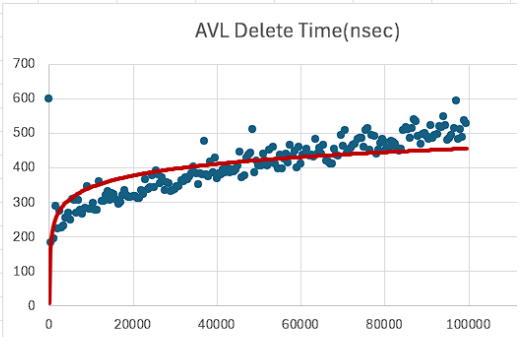
Zadanie 5 - Vyhľadávanie v dynamických množinách

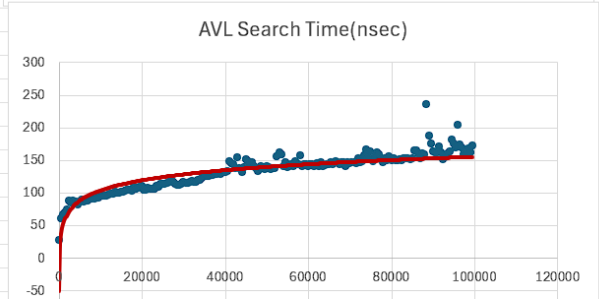
1. Implementácia AVL binárneho vyhľadávacieho stromu

AVL strom je samovyvažovací binárny vyhľadávací strom, kde rozdiel výšok ľavého a pravého podstromu každého uzla nie je väčší ako 1.
Po vložení alebo odstránení prvkov sa strom môže nevyvážiť a potom sa vykonajú rotácie na obnovenie rovnováhy.
Vďaka tomu majú operácie vyhľadávania, vkladania a mazania časovú zložitosť O(log n).

Nižšie sú zobrazené grafy priebehu troch funkcií algoritmu pre 200 rôznych prípadov. Funkcie sú vyjadrené ako Time(n), kde Time je čas v nanosekundách a n je počet prvkov. Červená krivka na grafoch predstavuje logaritmický trend závislosti.





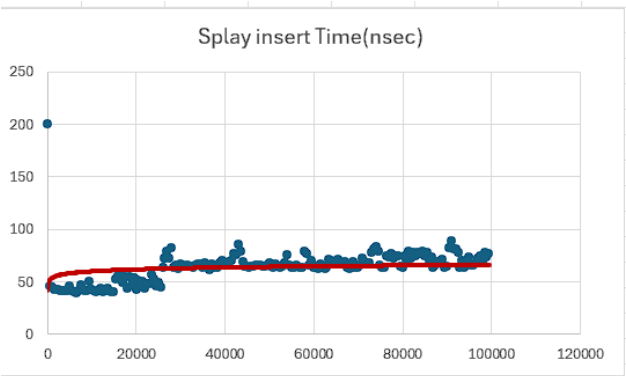


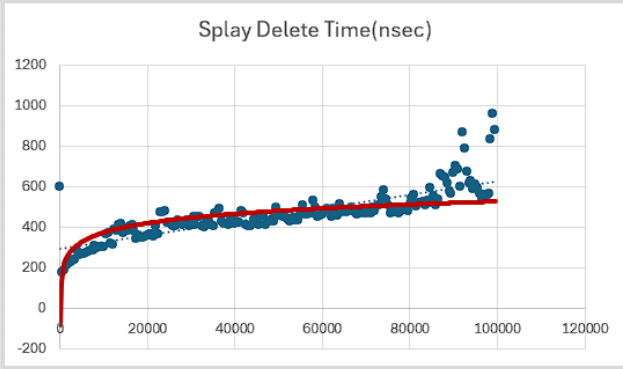
Ako môžeme vidieť z grafov, časová zložitosť každej z troch funkcií má logaritmický charakter, z čoho môžeme vyvodiť, že čas ich vykonania je O(log n).

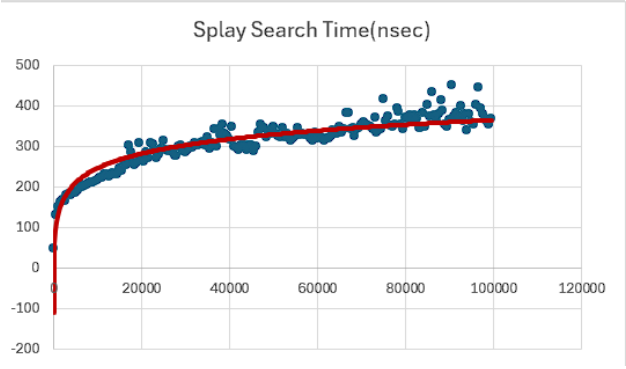
AVL operacia	Očakavana ČZ	ČZ implementacií
Insert	O(log(n))	O(log(n))
Search	O(log(n))	O(log(n))
Delete	O(log(n))	O(log(n))

2. Implementácia Splay binárneho vyhľadávacieho stromu

Splay strom je samooptimalizujúci binárny vyhľadávací strom, v ktorom sa pri každom prístupe k prvku vykoná operácia „splay“ (presun uzla do koreňa).
Tento presun sa realizuje pomocou rotácií (zig, zig-zig, zig-zag), čím sa často používané prvky dostávajú bližšie ku koreňu.
Vďaka tomu je amortizovaná časová zložitosť operácií vyhľadávania, vkladania a mazania O(log n).





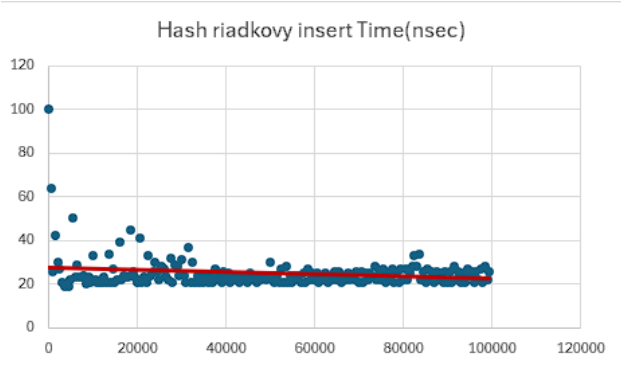


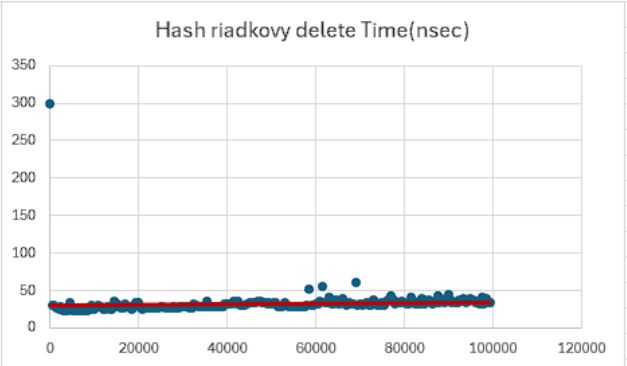
Každá operácia insert je taká rýchla (~60 nanosekúnd), že kvôli obmedzenej presnosti knižnice time.h nie je možné presne zmerať jej časovú zložitosť.
Ostatné dve funkcie majú podľa grafov logaritmický priebeh, čo naznačuje časovú zložitosť O(log n).

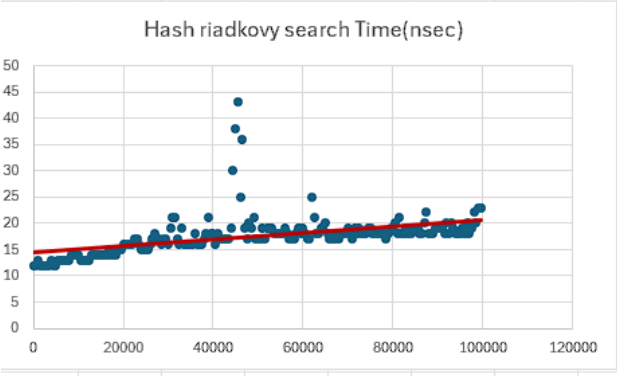
Splay operacia	Očakavana ČZ	ČZ implementacií
Insert	O(log(n))	Nemožne zmerať
Search	O(log(n))	O(log(n))
Delete	O(log(n))	O(log(n))

3. Implementácia hašovacej tabuľky s reťazením

Hašovacia tabuľka s reťazením ukladá prvky do poľa, kde každý index obsahuje zoznam (reťazec) prvkov s rovnakým hashom.
Pri kolízii (keď majú dva prvky rovnaký index) sa nový prvok jednoducho pridá do zoznamu na danom mieste.
Priemerná časová zložitosť operácií vyhľadávania, vkladania a mazania je O(1).





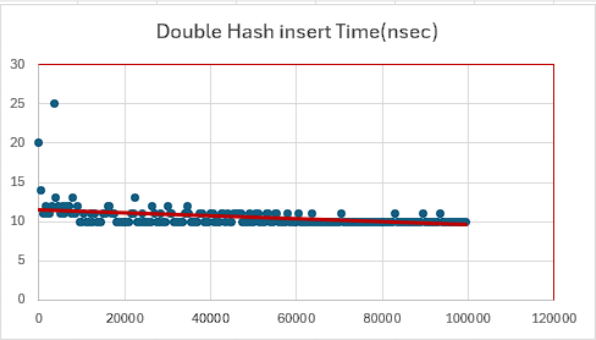


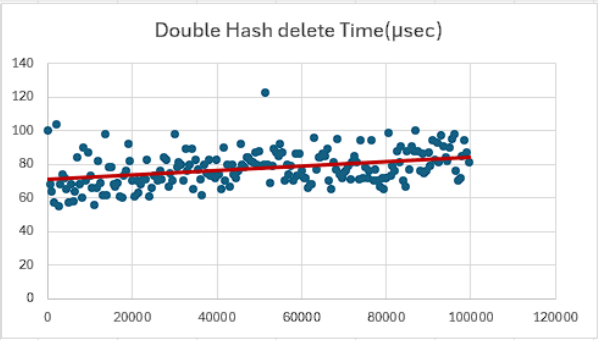
Ako môžeme vidieť z grafov, približný čas vykonania každej operácie je O(1), čo zodpovedá očakávaniam.

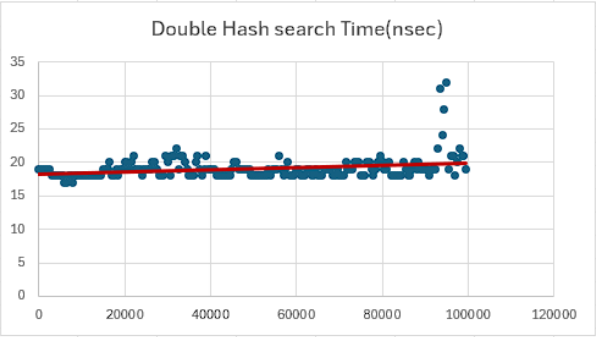
Hash table operation	Očakavana ČZ	ČZ implementacií
Insert	O(1)	O(1)
Search	O(1)	O(1)
Delete	O(1)	O(1)

4. Implementácia hašovacej tabuľky s dvojitým rozptýlením

Hašovacia tabuľka s dvojitým rozptýlením rieši kolízie pomocou druhej hash funkcie, ktorá určuje krok pri hľadaní voľného miesta.
Pri kolízii sa neukladajú prvky do zoznamu, ale hľadá sa ďalšia pozícia podľa vzorca h(i) = (h1(key) + i * h2(key)) % size.
Priemerná časová zložitosť operácií je O(1), v najhoršom prípade O(n).







Ako môžeme vidieť z grafov, približný čas vykonania každej operácie je O(1), čo zodpovedá očakávaniam.

Hash table operation	Očakavana ČZ	ČZ implementacií
Insert	O(1)	O(1)
Search	O(1)	O(1)
Delete	O(1)	O(1)